

Problem Analysis

การวิเคราะห์ปัญหา: Emergency Call Dispatch Queue

1. Problem Statement

ศูนย์รับแจ้งเหตุฉุกเฉินได้รับแจ้งเหตุหลายรายการเข้ามาอย่างต่อเนื่อง เช่น อุบัติเหตุ ไฟไหม้ เหตุเจ็บป่วย และเหตุทั่วไป โดยแต่ละเหตุการณ์มีระดับความรุนแรงแตกต่างกัน

ปัญหา คือ หากจัดการเหตุการณ์ตามลำดับเวลาที่รับแจ้งเพียงอย่างเดียว เหตุการณ์ที่มีความรุนแรงสูงอาจต้อง เหตุการณ์ที่มีความรุนแรงต่ำกว่า ดังนั้นจึงต้องออกแบบระบบ Queue เพื่อจัดลำดับการประมวลผลเหตุการณ์ และเปรียบเทียบ 2 วิธี ได้แก่ FIFO Queue และ Priority Queue

2. Input

ข้อมูลที่ระบบได้รับการแจ้งเหตุ ประกอบด้วย

Attribute	รายละเอียด
Event ID	รหัสเหตุการณ์
Event Type	ประเภทของเหตุการณ์
Time	เวลาที่ได้รับแจ้ง
Severity	ระดับความรุนแรง
Location	สถานที่เกิดเหตุ

ตัวอย่างระดับความรุนแรง

Severity	Priority
Critical	1
High	2
Medium	3
Low	4

หมายเหตุ: ตัวเลข Priority ยิ่งน้อย หมายถึงความสำคัญยิ่งสูง

3. Output

ระบบต้องแสดงลำดับการประมวลผล Event ที่ออกจาก Queue พร้อมข้อมูลสำคัญ ได้แก่

- Event ID
- Event Type
- Severity
- Location
- ลำดับการประมวลผล

และเปรียบเทียบผลลัพธ์ระหว่าง

- FIFO Event Queue
- Priority Event Queue

4. Constraints

1. Event ID ต้องไม่ซ้ำกัน
2. Event แต่ละรายการต้องมีข้อมูลครบถ้วน
3. Severity มี 4 ระดับ ได้แก่ Critical, High, Medium และ Low
4. FIFO Queue ต้องประมวลผลตามลำดับเวลาที่รับแจ้ง
5. Priority Queue ต้องประมวลผลเหตุการณ์ที่มีความรุนแรงสูงก่อน
6. หาก Event มี Priority เท่ากัน ให้พิจารณาลำดับเวลาที่รับแจ้ง โดย Event ที่เข้ามาก่อนต้องได้รับการประมวลผลก่อน
7. ระบบต้องรองรับกรณี Queue ว่าง
8. ระบบต้องสามารถยกเลิก Event ที่อยู่ใน Queue ได้

5. Queue เก็บข้อมูลอะไร

Queue จะเก็บข้อมูลประเภท **Event** โดยแต่ละ Event ประกอบด้วย

Event ID
Event Type
Time
Severity
Location

ตัวอย่าง

E1 เจ็บป่วย 10:01 Medium อาคาร A
E2 เหตุทั่วไป 10:02 Low ถนน B
E3 อุบัติเหตุ 10:03 Critical ถนน C

6. Queue ชนิดใดเหมาะสม

Algorithm A: FIFO Queue

ใช้ `ArrayDeque<Event>` เพราะต้องการจัดการเหตุการณ์ตามลำดับที่เข้ามาก่อน-หลัง

E1 → E2 → E3 → E4

จะประมวลผลเป็น

E1 → E2 → E3 → E4

Algorithm B: Priority Queue

ใช้ `PriorityQueue<Event>` เพราะต้องการจัดลำดับเหตุการณ์ตามระดับความรุนแรง

Critical → High → Medium → Low

ตัวอย่าง

E1 Medium

E2 Low

E3 Critical

E4 High

จะถูกประมวลผลเป็น

E3 → E4 → E1 → E2

7. เหตุผลที่ต้องใช้ Queue

ระบบ Emergency Call Dispatch มีลักษณะเป็นข้อมูลที่เข้ามาอย่างต่อเนื่องและมีเหตุการณ์จำนวนมากรอการประมวลผล จึงเหมาะกับโครงสร้างข้อมูลแบบ Queue

FIFO Queue เหมาะกับกรณีที่ต้องการความยุติธรรมตามลำดับเวลาที่แจ้งเหตุ ส่วน Priority Queue เหมาะกับสถานการณ์ฉุกเฉินจริง เพราะสามารถให้เหตุการณ์ที่มีความรุนแรงสูงได้รับการจัดการก่อนเหตุการณ์ทั่วไป

ดังนั้น การเปรียบเทียบทั้งสอง Algorithm จะช่วยให้สามารถเลือกโครงสร้างข้อมูลที่เหมาะสมกับระบบ Emergency Call Dispatch ได้อย่างมีประสิทธิภาพ